

EE 372 TERM PROJECT REPORT

Detection under QPSK Modulation with Gaussian Noise

Boran Aybak Kılıç  - 2022401207

Department of Electrical and Electronics Engineering - Bogazici University

June 2025

Contents

1	Introduction	2
2	System Model and Setup	2
2.1	QPSK Constellation	2
2.2	Noise Model	2
3	Simulation Approach	3
3.1	Overall Simulation Structure	3
3.2	Symbol Generation	3
3.3	Noise Generation and Signal Corruption	4
4	Detection Methods Implementation	4
4.1	Minimum Distance (MD) Detection	4
4.2	Maximum Likelihood (ML) Detection	5
4.3	Maximum A Posteriori (MAP) Detection	6
5	Performance Evaluation	7
5.1	Mean Squared Error Calculation	7
5.2	Simulation Results	8
6	Results Analysis and Discussion	9
6.1	SNR Dependency	9
6.2	Performance Comparison	9
6.2.1	Case 1: Equal Variance Noise	10
6.2.2	Case 2: Unequal Variance Noise	10
6.3	Theoretical MSE for ML and MD	11
6.4	Effect of Prior Probabilities	11
6.5	Computational Complexity	12
7	Conclusion	12
8	References	12
9	Supplementary	12

1 Introduction

This project implements and evaluates three fundamental detection strategies for Quadrature Phase Shift Keying (QPSK) modulation in the presence of additive Gaussian noise. The three detection methods analyzed are:

- **Minimum Distance Detection (MD)**: Chooses the constellation point with minimum Euclidean distance
- **Maximum Likelihood Detection (ML)**: Maximizes the likelihood function $p(y|x)$
- **Maximum A Posteriori Detection (MAP)**: Maximizes the posterior probability $p(x|y)$

The performance of each method is evaluated using Mean Squared Error (MSE) under two different noise variance scenarios across various Signal-to-Noise Ratio (SNR) levels.

2 System Model and Setup

2.1 QPSK Constellation

The QPSK constellation consists of four equally spaced points on the unit circle. We can then encode the symbols as complex numbers whose real and imaginary parts are comprised of in-phase and quadrature components, respectively.

Specifically, the constellation points are:

$$S_1 = \frac{1}{\sqrt{2}} + j\frac{1}{\sqrt{2}} \quad (1)$$

$$S_2 = \frac{1}{\sqrt{2}} - j\frac{1}{\sqrt{2}} \quad (2)$$

$$S_3 = -\frac{1}{\sqrt{2}} + j\frac{1}{\sqrt{2}} \quad (3)$$

$$S_4 = -\frac{1}{\sqrt{2}} - j\frac{1}{\sqrt{2}} \quad (4)$$

The symbols are transmitted with non-uniform prior probabilities:

$$P(S_1) = 0.4 \quad (5)$$

$$P(S_2) = 0.3 \quad (6)$$

$$P(S_3) = 0.2 \quad (7)$$

$$P(S_4) = 0.1 \quad (8)$$

which will affect the decision boundaries in the MAP detection.

2.2 Noise Model

Noise in the communication channel is assumed to be additive white Gaussian noise (AWGN) whose characteristics in time are independent/uncorrelated of each other, and it has a flat frequency spectrum. Two noise scenarios are considered:

Case 1: Equal Variance Noise

$$\sigma_I^2 = \sigma_Q^2 = \frac{1}{2 \cdot \text{SNR}_{\text{linear}}} \quad (9)$$

Case 2: Unequal Variance Noise

$$\sigma_I^2 = \frac{2}{3 \cdot \text{SNR}_{\text{linear}}} \quad (10)$$

$$\sigma_Q^2 = \frac{1}{3 \cdot \text{SNR}_{\text{linear}}} \quad (11)$$

where $\text{SNR}_{\text{linear}} = 10^{\text{SNR}_{\text{dB}}/10}$ and SNR values tested are $[-10, -5, 0, 5, 10, 15]$ dB.

3 Simulation Approach

3.1 Overall Simulation Structure

The simulation is structured to systematically evaluate all detection methods across both noise scenarios. The key implementation details are shown below:

```

1 % QPSK constellation points (double precision)
2 constellation = double([1/sqrt(2) + 1j/sqrt(2);    % S1
3                     1/sqrt(2) - 1j/sqrt(2);    % S2
4                     -1/sqrt(2) + 1j/sqrt(2);    % S3
5                     -1/sqrt(2) - 1j/sqrt(2)]); % S4
6
7 % Prior probabilities for MAP detection
8 prior_probs = double([0.4; 0.3; 0.2; 0.1]);
9
10 % System parameters
11 N_symbols = int32(1e5); % Number of symbols
12 Es = double(1); % Average symbol energy
13 SNR_dB = double([-10, -5, 0, 5, 10, 15]); % SNR values in dB
14 SNR_linear = double(10.^(SNR_dB/10)); % Convert to linear scale

```

Listing 1: System Parameters Setup

3.2 Symbol Generation

Random QPSK symbols are generated according to the specified prior probabilities using inverse transform sampling. First, the cumulative distribution of prior probabilities is calculated as $[0.4, 0.7, 0.9, 1.0]$. Second, random samples are generated from a *Uniform(0,1)* distribution. Lastly, a symbol is assigned to each sample depending on which interval its value falls within. In this manner, we ensure that 40% of the symbols generated are S_1 , ..., and 10% of the symbols generated are S_4 . Symbols are represented with $i \in \{1, 2, 3, 4\}$.

```

1 % Generate random symbols according to prior probabilities
2 cumulative_probs = double(cumsum(prior_probs));
3 random_vals = double(rand(N_symbols, 1));
4
5 % Map random values to constellation indices
6 symbol_indices = ones(N_symbols, 1);

```

```

7 for i = 1:length(cumulative_probs)
8     symbol_indices(random_vals > cumulative_probs(i)) = int32(i + 1);
9 end
10
11 % Get the actual transmitted symbols
12 transmitted_symbols = double(constellation(symbol_indices));

```

Listing 2: Symbol Generation with Prior Probabilities

3.3 Noise Generation and Signal Corruption

For each SNR level and noise case (equal and unequal variances), independent Gaussian noise is added to the I and Q components. We use MATLAB's *randn()* function to draw a sample from the normal distribution and scale it with the corresponding standard deviation σ , since the noise is white Gaussian and is independent of other time instances at each time step (or sample). This operation is equivalent to drawing samples from a Gaussian random variable with zero mean and variance σ^2 . Also note that the double function is used for higher precision.

```

1 % Generate noise (double precision)
2 noise_I = double(sqrt(sigma2_I) * randn(N_symbols, 1));
3 noise_Q = double(sqrt(sigma2_Q) * randn(N_symbols, 1));
4 noise = double(noise_I + 1j*noise_Q);
5
6 % Received symbols (double precision)
7 received_symbols = double(transmitted_symbols + noise);

```

Listing 3: Noise Generation and Signal Corruption

4 Detection Methods Implementation

4.1 Minimum Distance (MD) Detection

The MD detector selects the constellation point that minimizes the Euclidean distance to the received signal. Since each received signal is a complex number corrupted by Gaussian noise in both in-phase and quadrature components, we can calculate the length of the difference vector from each of the 4 constellation points using the *abs()* function, which works with complex numbers as well.

$$\hat{x} = \arg \min_{\hat{x} \in S} \|y - x\| \quad (12)$$

```

1 function detected_idx = minimum_distance_detection(received,
2     constellation)
3     N = length(received);
4     detected_idx = zeros(N, 1);
5
6     for i = 1:N
7         % Calculate squared Euclidean distances to all constellation
8         % points
9         distances = double(abs(received(i) - constellation));
10
11         % Find the index of minimum distance

```

```

10         [~, detected_idx(i)] = min(distances);
11     end
12 end

```

Listing 4: Minimum Distance Detection Implementation

4.2 Maximum Likelihood (ML) Detection

The ML detector maximizes the likelihood function $p(y|x)$, considering the noise statistics:

$$\hat{x} = \arg \max_{\hat{x} \in S} p(y|x) \quad (13)$$

For AWGN with independent I and Q components:

$$p(y|x) = p(y_I|x_I) \times p(y_Q|x_Q) \quad (14)$$

where each component follows a Gaussian distribution with mean x_I or x_Q and standard deviation σ_I or σ_Q :

$$p(y_I|x_I) = \frac{1}{\sqrt{2\pi\sigma_I^2}} \exp\left(-\frac{(y_I - x_I)^2}{2\sigma_I^2}\right), \quad p(y_Q|x_Q) = \frac{1}{\sqrt{2\pi\sigma_Q^2}} \exp\left(-\frac{(y_Q - x_Q)^2}{2\sigma_Q^2}\right) \quad (15)$$

$$p(y|x) = \frac{1}{2\pi\sigma_I\sigma_Q} \exp\left(-\left(\frac{(y_I - x_I)^2}{2\sigma_I^2} + \frac{(y_Q - x_Q)^2}{2\sigma_Q^2}\right)\right) \quad (16)$$

Since the natural logarithm is a monotonically increasing function, we can alternatively maximize $\log(p(y|x))$, which is called the log-likelihood. Furthermore, instead of maximizing the log-likelihood, we can minimize the negative log-likelihood, which is a common practice utilized in machine learning, ensuring numerical stability when dealing with small probabilities and providing ease of calculation due to the multiplicative nature of independent random variables.

$$\hat{x} = \arg \min_{\hat{x} \in S} [-\log(p(y|x))] \quad (17)$$

$$= \arg \min_{\hat{x} \in S} \log(2\pi\sigma_x\sigma_y) + \frac{(y_I - x_I)^2}{2\sigma_I^2} + \frac{(y_Q - x_Q)^2}{2\sigma_Q^2} \quad (18)$$

$$= \arg \min_{\hat{x} \in S} \frac{(y_I - x_I)^2}{2\sigma_I^2} + \frac{(y_Q - x_Q)^2}{2\sigma_Q^2} \quad (19)$$

```

1 function detected_idx = maximum_likelihood_detection(received,
2   constellation, sigma2_I, sigma2_Q)
3     N = length(received);
4     detected_idx = zeros(N, 1);
5
6     for i = 1:N
7         % Extract I and Q components of received symbol
8         y_I = double(real(received(i)));
9         y_Q = double(imag(received(i)));

```

```

9
10 % Calculate log-likelihood for numerical stability
11 log_likelihoods = double(zeros(4, 1));
12 for k = 1:4
13     % Extract I and Q components of constellation point
14     x_I = double(real(constellation(k)));
15     x_Q = double(imag(constellation(k)));
16
17     % Calculate log-likelihood components
18
19     neg_log_like_I = double(0.5*log(2*pi*sigma2_I) + (y_I -
20         x_I)^2 / (2*sigma2_I));
21
22     neg_log_like_Q = double(0.5*log(2*pi*sigma2_Q) + (y_Q -
23         x_Q)^2 / (2*sigma2_Q));
24
25     % Total log-likelihood: log p(y|x) = log p(y_I|x_I) + log
26     % p(y_Q|x_Q)
27     neg_log_likelihoods(k) = double(neg_log_like_I +
28         neg_log_like_Q);
29
30 end
31 end

```

Listing 5: Maximum Likelihood Detection Implementation

4.3 Maximum A Posteriori (MAP) Detection

The MAP detector maximizes the posterior probability $p(x|y)$ using Bayes' theorem and the fact that multiplying by the constant factor $P(y)$ does not change the quantity to be maximized:

$$\hat{x} = \arg \max_{x \in S} p(x|y) = \arg \max_{x \in S} p(x, y) = \arg \max_{x \in S} p(y|x) \cdot P(x) \quad (20)$$

This incorporates both the likelihood function and the prior probabilities.

$$\hat{x} = \arg \max_{\hat{x} \in S} \frac{p(y)}{2\pi\sigma_I\sigma_Q} \exp \left(- \left(\frac{(y_I - x_I)^2}{2\sigma_I^2} + \frac{(y_Q - x_Q)^2}{2\sigma_Q^2} \right) \right) \quad (21)$$

$$= \arg \min_{\hat{x} \in S} \left(0.5 \log(2\pi\sigma_I) + \frac{(y_I - x_I)^2}{2\sigma_I^2} \right) + \left(0.5 \log(2\pi\sigma_Q) + \frac{(y_Q - x_Q)^2}{2\sigma_Q^2} \right) - \log(p(y)) \quad (22)$$

$$= \arg \min_{\hat{x} \in S} \frac{(y_I - x_I)^2}{2\sigma_I^2} + \frac{(y_Q - x_Q)^2}{2\sigma_Q^2} - \log(p(y)) \quad (23)$$

```

1 function detected_idx = maximum_aposteriori_detection(received,
2   constellation, sigma2_I, sigma2_Q, priors)
   N = length(received);

```

```

3 detected_idx = zeros(N, 1);
4
5 for i = 1:N
6     % Extract I and Q components of received symbol
7     y_I = double(real(received(i)));
8     y_Q = double(imag(received(i)));
9
10    % Calculate log-posterior for numerical stability
11    log_posteriors = double(zeros(4, 1));
12    for k = 1:4
13        % Extract I and Q components of constellation point
14        s_I = double(real(constellation(k)));
15        s_Q = double(imag(constellation(k)));
16
17        % Calculate log-likelihood components
18        neg_log_like_I = double(0.5*log(2*pi*sigma2_I) + (y_I -
19            s_I)^2 / (2*sigma2_I));
20        neg_log_like_Q = double(0.5*log(2*pi*sigma2_Q) + (y_Q -
21            s_Q)^2 / (2*sigma2_Q));
22
23        % Total log-likelihood
24        neg_log_likelihood = double(neg_log_like_I +
25            neg_log_like_Q);
26
27        % Log-posterior: log p(x|y) = log p(y|x) + log P(x) +
28            constant
29        log_posteriors(k) = double(neg_log_likelihood -
30            log(priors(k)));
31    end
32 end

```

Listing 6: Maximum A Posteriori Detection Implementation

5 Performance Evaluation

5.1 Mean Squared Error Calculation

The performance metric used is the Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \|\hat{x}_i - x_i\|^2 \quad (24)$$

where $\hat{x}_i$ is the detected symbol and x_i is the transmitted symbol. Note that we consider the constellation of the detected symbol $\hat{x} \in \{\pm \frac{1}{\sqrt{2}} \pm i \frac{1}{\sqrt{2}}\}$, not the received signal. If symbols in QPSK are encoded using Gray coding (where each adjacent symbol has a 1-bit difference), MSE corresponds to the bit error rate (BER), which is a popular evaluation metric used in digital communications, since $\|x_{00} - x_{01}\|^2 = \|x_{00} - x_{10}\|^2 = \frac{\|x_{00} - x_{11}\|^2}{2}$.

```

1 % Calculate MSE for each method

```

```

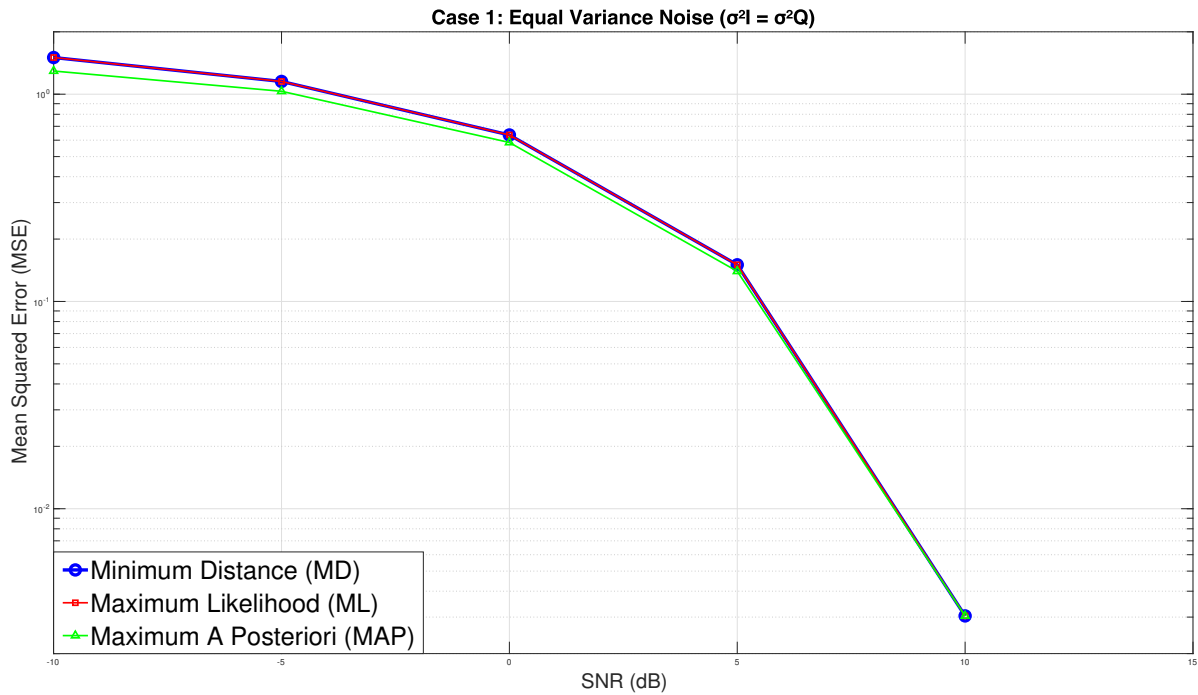
2 MSE_MD = double(mean(abs(constellation(detected_MD) -
    transmitted_symbols).^2));
3 MSE_ML = double(mean(abs(constellation(detected_ML) -
    transmitted_symbols).^2));
4 MSE_MAP = double(mean(abs(constellation(detected_MAP) -
    transmitted_symbols).^2));

```

Listing 7: MSE Calculation

5.2 Simulation Results

The simulation generates the following MSE vs. SNR plots for both noise cases, comparing all three detection methods.

Figure 1: MSE vs SNR for equal noise variances $\sigma_I^2 = \sigma_Q^2$

It is important to make the following remarks:

- In case it is not apparent to the reader, ML and MD performance in both cases are identical and thus, their curves are plotted coincidentally.
- MSE for all 3 detection methods in both equal and unequal noise variance cases are computed to be zero. Although, theoretically, absolute zero MSE is not feasible for AWGN channels, the probability of misdetection is so low that 10^5 samples are not enough to capture even a single erroneous symbol.
- Although it was instructed that "Set the range of y-axes no wider than $(10^{-5}, 1)$ ", MSE is computed to be larger than 1 for SNR = -10, -5 in both equal and unequal noise variance cases. Thus, the limits of the y-axis are set to $(2 \cdot 10^{-3}, 2)$.

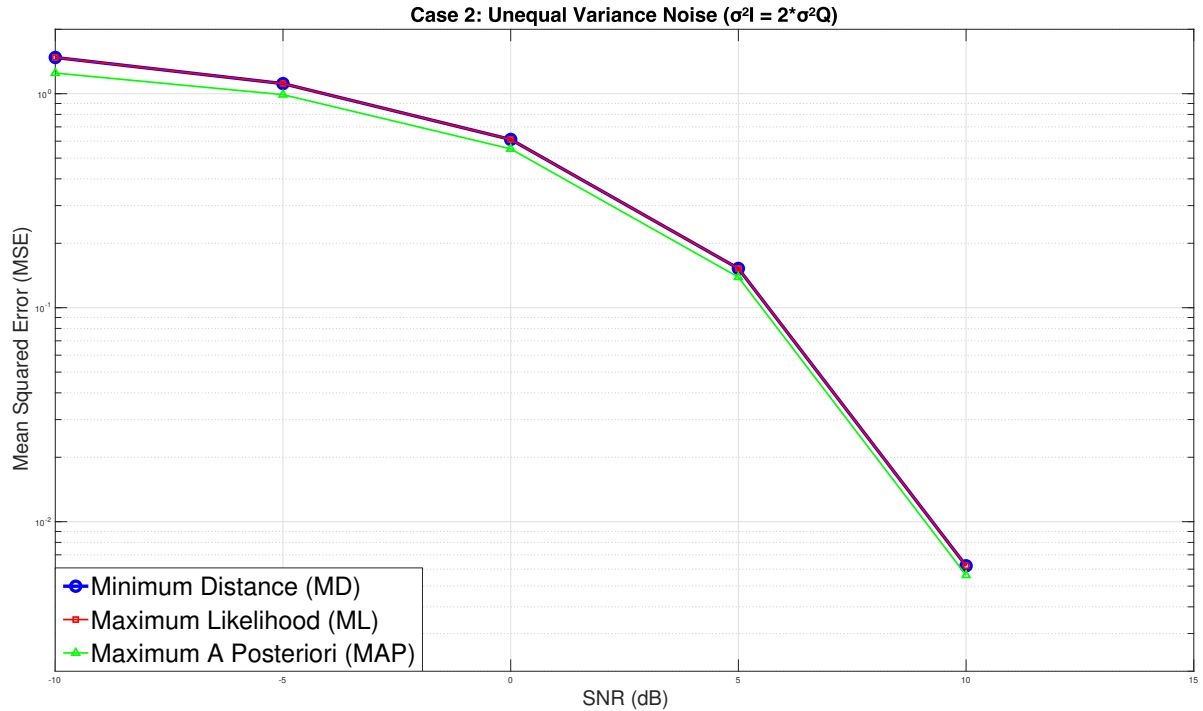


Figure 2: MSE vs SNR for unequal noise variances $\sigma_I^2 = 2\sigma_Q^2$

6 Results Analysis and Discussion

6.1 SNR Dependency

All detectors show improved performance (lower MSE) as SNR increases. This is expected since at low SNR, noise is more pronounced and received signals are distributed over a larger region in the constellation space. This results in more erroneous symbol detections compared to high SNR, where the noisy signal distribution is more tightly bound around the symbols $S_1, \dots, S_4$. Also, at high SNR values, the differences between detection methods become less significant as noise becomes negligible compared to signal power, which is a common phenomenon in digital communications.

6.2 Performance Comparison

Based on the simulation results, several key observations can be made. Firstly, the performance of the detection methods does not show a significant difference between the two cases since the average symbol energy $E_s = 1$ and thus, the total noise power is the same. However, slight variations (within the same order or maybe even less) occur, with MSE for equal noise variance being slightly lower. It is assumed that the slight reduction in MSE for equal noise variance is due to the fact that either the I or Q component noise becomes a bottleneck with unequal noise variance, leading to higher MSE, whereas equal noise variance leads to safer circular distributions. Nevertheless, this discussion merits more detailed research on its own, which is out of the scope of this project.

6.2.1 Case 1: Equal Variance Noise

When the I and Q components have equal noise variances, the ML and MD detectors perform identically. This is because the optimal decision boundaries become circular around each constellation point, making the Euclidean distance criterion equivalent to the likelihood maximization criterion. Using 4.2 with $\sigma_I = \sigma_Q = \sigma$,

$$\hat{x}_{ML} = \arg \min_{\hat{x} \in S} \frac{(y_I - x_I)^2}{2\sigma^2} + \frac{(y_Q - x_Q)^2}{2\sigma^2} \quad (25)$$

$$= \arg \min_{\hat{x} \in S} (y_I - x_I)^2 + (y_Q - x_Q)^2 \quad (26)$$

$$= \arg \min_{\hat{x} \in S} \|y - x\|^2 = \arg \min_{\hat{x} \in S} \|y - x\| \quad (27)$$

We see that ML detection is theoretically equivalent to MD detection, confirming our simulation results. MAP detection provides superior performance compared to ML and MD since it incorporates the prior probabilities into decision thresholds. Decision boundaries for MD, ML, and MAP detection can be observed from Fig. 3, 5, and 4 in Section 9. These plots show the received signals' I and Q components in the 2D complex plane and what symbol they are detected as. We see that decision boundaries for MD and ML are simply the 4 quadrants of the complex plane, while MAP offers a larger margin of error for S_1 since it is more probable that the received signal was in fact transmitted as S_1 , maximizing the a posteriori probability.

6.2.2 Case 2: Unequal Variance Noise

When the noise variances are unequal ($\sigma_I^2 \neq \sigma_Q^2$), one would expect the ML detector to outperform the MD detector since the ML detector takes into account noise statistics as well. However, we see that the performance of ML and MD schemes is still exactly the same 2. The ML detector accounts for the different noise statistics in I and Q channels, resulting in elliptical decision boundaries (with major/minor axis lengths σ_I/σ_Q and centered at (x_I, x_Q)) that better match the noise characteristics.

$$\hat{x}_{ML} = \arg \min_{\hat{x} \in S} \frac{(y_I - x_I)^2}{2\sigma_I^2} + \frac{(y_Q - x_Q)^2}{2\sigma_Q^2} \quad (28)$$

In general, ML detection will, in fact, perform better than MD. However, in QPSK, since constellation symbols are symmetric with respect to the x- and y-axes, decision regions are still the 4 quadrants of the complex plane, as in MD detection. To better comprehend why this is true, one can think of ML detection as squeezing or expanding the x- and y-axes with factors σ_I, σ_Q before implementing MD detection. Since each constellation point S_1, S_2, S_3, S_4 is equally spaced in 4 quadrants, scaling of the axes will not affect how symbols are detected. Detection depends only on the sign of the I and Q components of the received signal. It will certainly be detected as the symbol whose signs of I and Q components correspondingly match. See Fig. 6 to see how the received signal is detected based on which of the 4 ellipses first encapsulates a given point. Note that these ellipses always intersect along the I and Q axes, which are, trivially, the decision boundaries.

6.3 Theoretical MSE for ML and MD

Since received symbols are normally distributed and decision boundaries for ML and MD detection are simply the 4 quadrants, it is rather simple to calculate the MSE as:

$$MSE = \sum_{i=1}^4 \sum_{j=1}^4 (x_i - x_j)^2 P(x_j|x_i) P(x_i) \quad (29)$$

$$= [2P(x_2|x_1) + 2P(x_3|x_1) + 4P(x_4|x_1)] \cdot 0.4 \quad (30)$$

$$+ [2P(x_1|x_2) + 2P(x_4|x_2) + 4P(x_3|x_2)] \cdot 0.3 \quad (31)$$

$$+ [2P(x_1|x_3) + 2P(x_4|x_3) + 4P(x_2|x_3)] \cdot 0.2 \quad (32)$$

$$+ [2P(x_2|x_4) + 2P(x_3|x_4) + 4P(x_1|x_4)] \cdot 0.1 \quad (33)$$

$$(34)$$

where $x_{i,j} \in \{\pm \frac{1}{\sqrt{2}} \pm i \frac{1}{\sqrt{2}}\}$ and $P(x_j|x_i)$ is the probability that x_j is detected given that x_i is transmitted. Expanding the conditional probabilities in terms of Φ or Q functions, we calculate the MSE values as given in the table below.

SNR (dB)	Equal Noise Variance	Unequal Noise Variance
-10	1.504305	1.484431
-5	1.146884	1.116403
0	0.634455	0.607897
5	0.150905	0.153050
10	0.003148	0.006395
15	0.000000	0.000000

Table 1: Theoretical MSE values under equal and unequal noise variance at different SNR levels

SNR (dB)	Equal Noise Variance	Unequal Noise Variance
-10	1.50385	1.48546
-5	1.15023	1.11706
0	0.63653	0.60753
5	0.14969	0.15283
10	0.00304	0.00682
15	0.00000	0.00000

Table 2: Experimental MSE values under equal and unequal noise variance at different SNR levels

We see that our simulation results very closely match the theoretical expectations. Also, MSE is calculated to be zero for SNR = 15 dB within MATLAB's precision limits, confirming that this behavior is not due to some mistake in our simulation implementation.

6.4 Effect of Prior Probabilities

The MAP detector incorporates prior symbol probabilities, which provide additional performance improvements, especially at lower SNR values where noise dominates. The

non-uniform priors favor more probable symbols, reducing overall error rates. If prior probabilities were equal, MAP detection would be equivalent to ML.

6.5 Computational Complexity

- **MD Detection:** Lowest complexity, requiring only distance calculations
- **ML Detection:** Moderate complexity, requiring likelihood computations with noise statistics
- **MAP Detection:** Highest complexity, requiring both likelihood and prior probability computations

7 Conclusion

This project successfully implemented and evaluated three fundamental detection strategies for QPSK modulation under Gaussian noise conditions. Key findings include:

1. MD and ML detectors perform identically under equal variance noise conditions
2. ML detection does not provides superior performance under unequal variance noise by accounting for different noise statistics in I and Q channels as one would generally expect since symbol constellation in QPSK is symetric with respect to I and Q axes.
3. MAP detection achieves the best overall performance by incorporating both noise statistics and prior symbol probabilities.
4. Performance improvements are most significant at lower SNR values where noise effects dominate.
5. The choice of detection method involves a trade-off between performance and computational complexity.

The simulation results demonstrate the theoretical predictions about optimal detection strategies in digital communication systems, confirming that the MAP detector provides the minimum probability of error when prior probabilities are known and properly incorporated.

8 References

1. Proakis, J. G., & Salehi, M. (2008). *Digital Communications* (5th ed.). McGraw-Hill.
2. Kay, S. M. (1998). *Fundamentals of Statistical Signal Processing: Detection Theory*. Prentice Hall.
3. Sklar, B. (2001). *Digital Communications: Fundamentals and Applications* (2nd ed.). Prentice Hall.

9 Supplementary

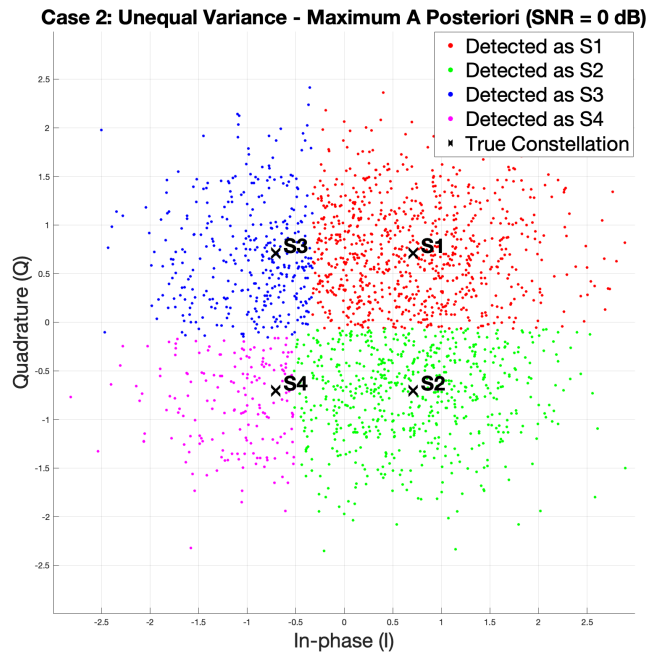


Figure 3: Decision Boundaries of MAP Detection at 0dB with Unequal Variance Noise

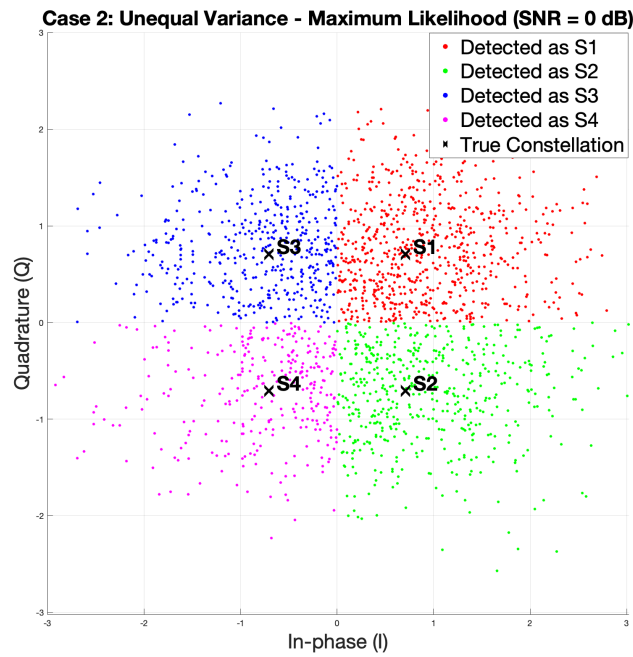


Figure 4: Decision Boundaries of ML Detection at 0dB with Unequal Variance Noise

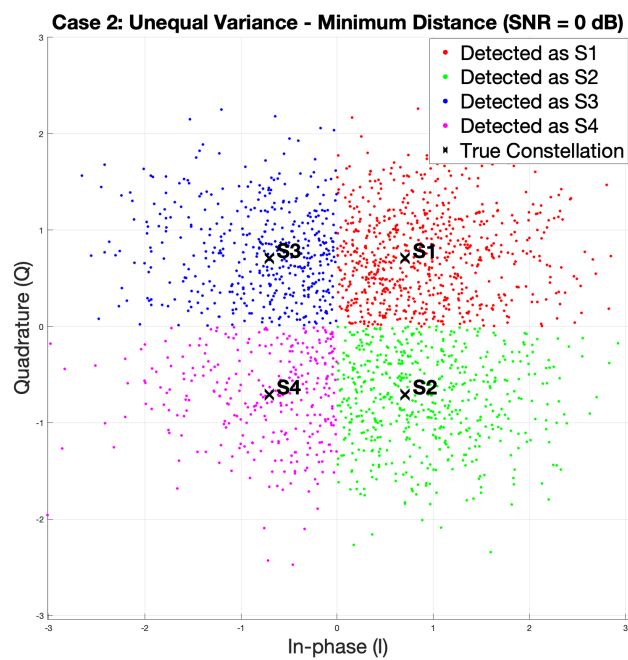
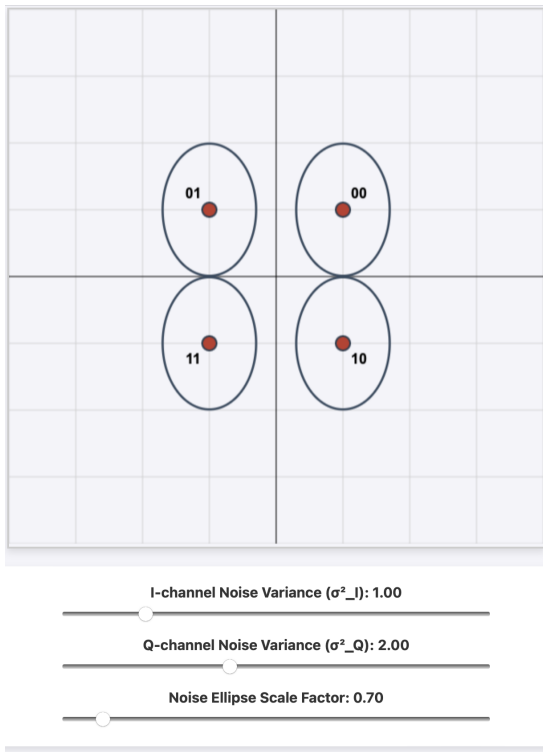
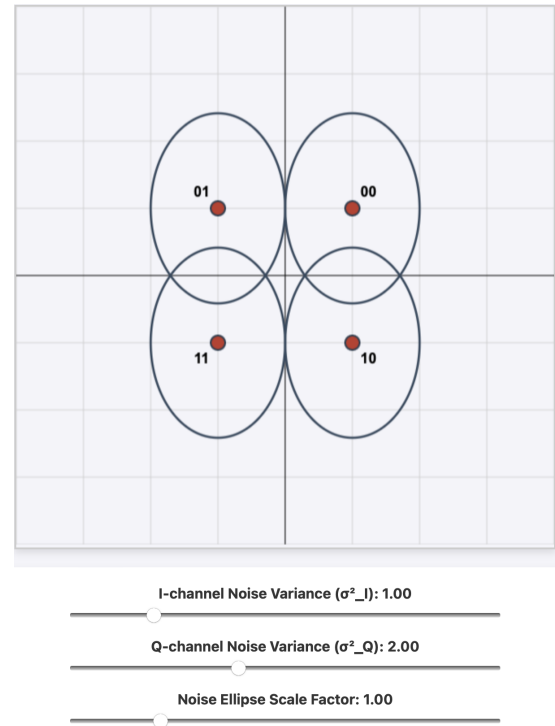


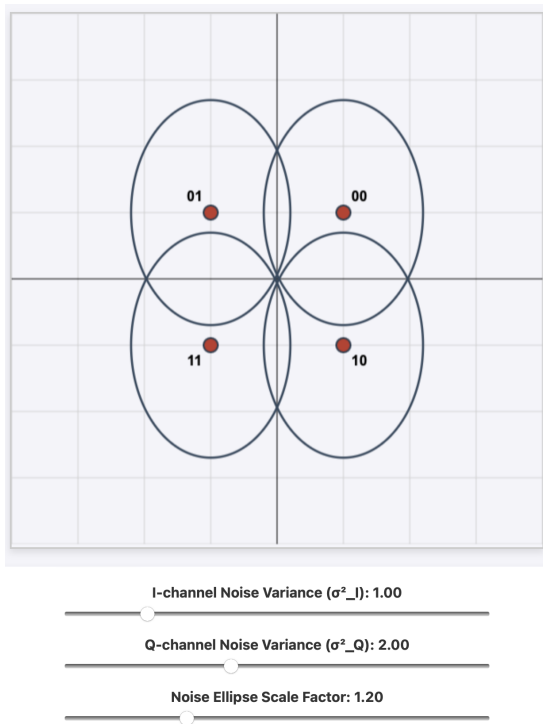
Figure 5: Decision Boundaries of MD Detection at 0dB with Unequal Variance Noise



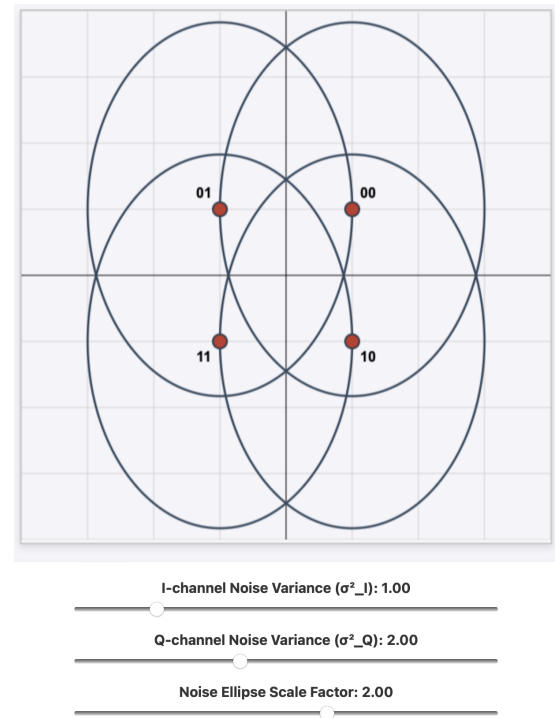
(a) Ellipse scale factor = 0.7



(b) Ellipse scale factor = 1.0



(c) Ellipse scale factor = 1.25



(d) Ellipse scale factor = 2.0

Figure 6: Effect of ellipse scaling on decision regions. These images were obtained from a custom website powered with a short HTML written by the author.